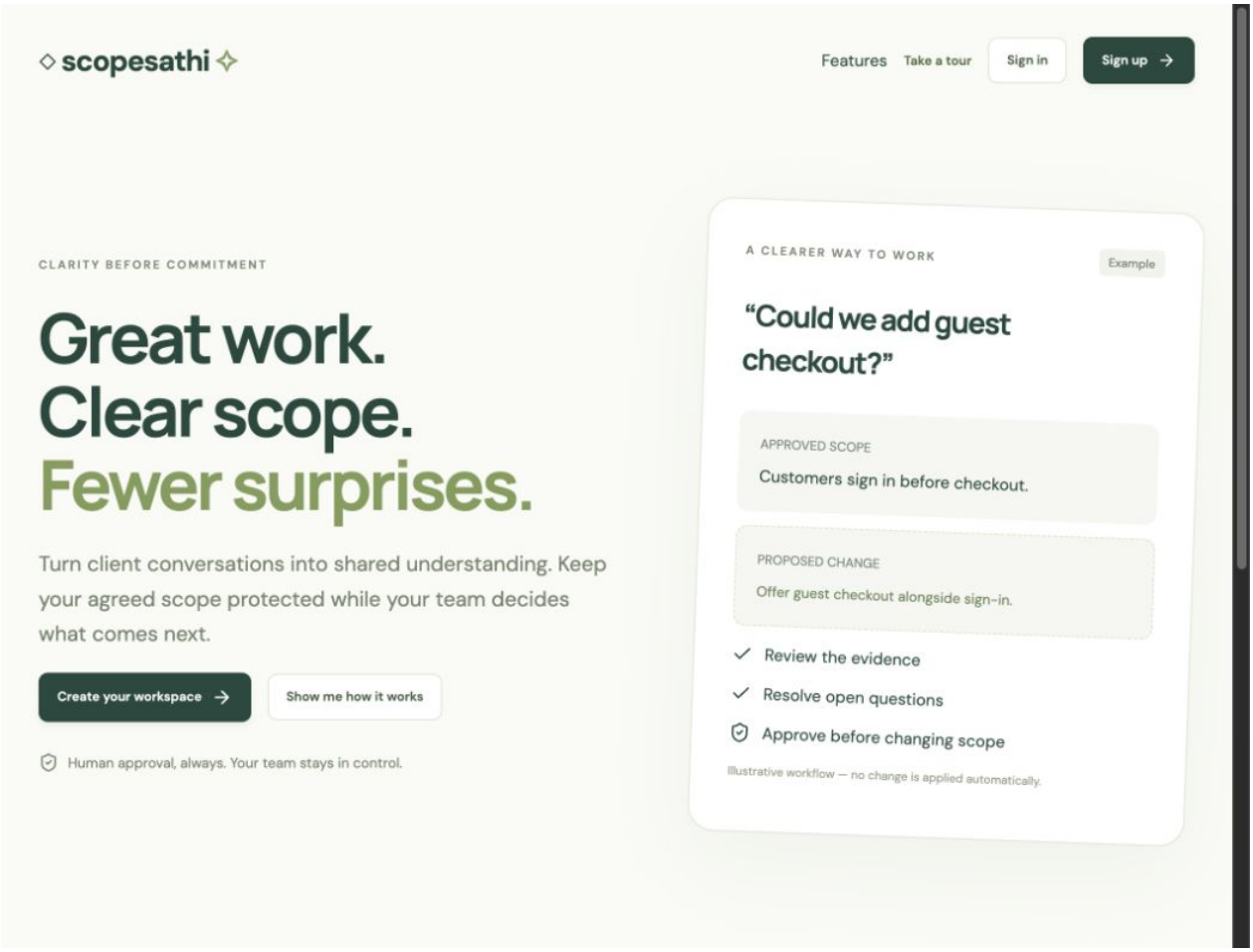
ScopeSathi

Clarity before commitment.

A client-requirements and scope-change workspace for freelancers and small software teams.



Current status: manual review workflows, revision pricing and real Firestore persistence are implemented and tested. Automatic Gemini analysis and public deployment remain pending.

This report explains the implementation and evidence boundaries. Historical UI captures show demonstration data; they are not proof of current cloud or AI connectivity.

Author: Shaurya Verma
Repository: https://github.com/shauryaverma03/Scope-Sathi

01 / What ScopeSathi does

A client agrees to a specific project, then changes requirements near delivery. ScopeSathi keeps the original agreement, identifies proposed changes, records a human decision and makes the impact on work and revision allowance visible.

Stage	User action and result
Client brief	Record structured intake or upload a selectable-text PDF/TXT. Retain the original brief text.
Reviewed baseline	Edit requirement wording and exact source quotes. A reviewer approves version 1.
Incoming request	Save the client message, source and revision round. Compare it against the latest approved scope.
Requirement comparison	Review IDs, original wording, exact request quotes, proposed wording, questions and linked tasks.
Human approval	Approve, reject or ask for clarification. Evidence, open questions, permissions and stale versions gate approval.
Task and cost impact	Preserve old versions, flag affected tasks, count approved rounds and obtain client consent for priced extra work.

ScopeSathi manages agreements about work. The demo online-store brief is test input; ScopeSathi does not build the store, implement its checkout or send its order-confirmation emails.

The accompanying manual guide provides copyable messages, expected outcomes and checks for the current manual-analysis mode.

02 / Intake, documents and approved scope

Feature	How it works
Structured client intake	Capture project goal, intended users, required features, deliverables, constraints, acceptance criteria and excluded work. Save separately or convert the intake into a brief.
Pasted brief / TXT / PDF	The backend reads selectable text. The current limits are 2 MB for uploads and 16,000 characters for a brief. Uploaded PDF bytes are not retained after extraction.
Scanned documents	Image-only PDF pages require OCR outside this application. OCR is not implemented.
Requirement editor	Review and edit extracted entries. Each requirement has a stable ID, editable wording and an exact source passage. Up to 30 requirement sections are supported.
Initial approval	Only a reviewer/owner approves the first baseline. Intake and original approved evidence are preserved.
Versions and comparison	Later approved scope changes create new snapshots. Stable IDs make additions, edits and removals visible.

Important limitation: with no Gemini key/model configured, extraction splits text lines rather than understanding requirements. Headings, metadata and wrapped lines may become entries. Review and consolidate them before approval.

Exact passages and reference checks prevent fabricated evidence from being accepted. They do not prove that a proposal is semantically correct; the human reviewer remains responsible.

The supplied brief has seven labelled feature/boundary clauses. The PDF labels R1-R7 are test references; use the actual IDs in your reviewed baseline.

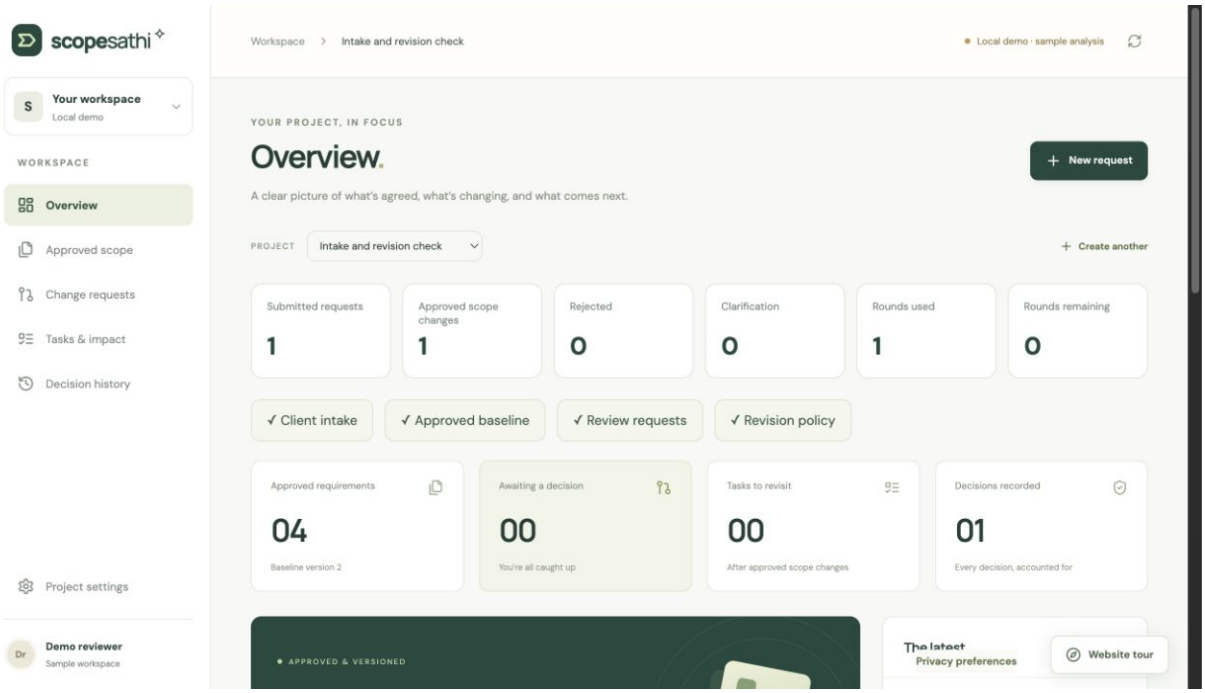
03 / Requests, decisions and task impact

Feature	Behavior
Request list	Submit a client message, title and source. Search and filter by status. Group related requests into a named round.
Finding types	Addition, modification, removal, clarification or no change. Multiple findings may belong to one request.
Affected requirements	A modification/removal must reference an approved requirement. Show original wording and exact new-request evidence.
Ambiguous matches	Keep clarification questions open until resolved. Approval is blocked while clarification findings/questions remain.
Manual / configured AI	Current mode creates a clearly labelled manual proposal. Configured Gemini can propose findings, which still need validation and human review.
Review outcomes	Approve, reject or request clarification with a reason. A clarification decision stays in history; clarified details become a follow-up request.
Version and concurrency checks	Old-baseline analysis and outdated project revisions are rejected. Refresh/reconcile rather than overwriting another edit.
Task impact	Tasks link to requirement IDs. Changed requirements flag linked tasks; explicitly selected impact links also flag work. Humans update task status after review.

Conflicting edits to the same requirement in one approval are blocked. Combine the intended replacement into one finding or split the work for review.

Scope/history export as JSON; tasks export as spreadsheet-friendly CSV. These are snapshots, not a complete database backup/restore mechanism.

04 / Workspace and onboarding



Historical application screenshot with demonstration data. Pricing controls were added later and are described on the next page.

Section	Purpose
Overview	Project summary, setup checklist, pending decisions, revision/cost meter and affected-task counts.
Approved scope	Client intake, documents, requirement review, baseline approval and version comparison.
Change requests	Search/status filters, proposals, review actions and cost consent.
Tasks / Decision history	Requirement-linked board; chronological reasons, reviewers, versions and exports.
Settings	Account, actual connection status, access, revision allowance, price and designated client. Account/connection settings also work before a project exists.

The public home page explains the workflow and offers Google sign-in/sign-up entry points and a tour. Google manages authentication; a new verified account becomes a first-time user. Each empty section has its own next action.

Session preferences, optional tour remembrance, profile/sign-out controls and branded loading transitions are implemented. Reduced-motion behavior is supported. A fresh full mobile/browser walkthrough was not completed in this run.

05 / Revision allowance and cost meter

The owner defines a revision **round**: related client requests reviewed together. Each approved new-scope round counts once. Corrections to meet the original agreement, rejected requests, clarification and no-change decisions do not consume allowance.

Example configuration	Illustrative values - not a live customer invoice
Included allowance	5 rounds
Additional round rate	INR 500 per extra round
Two additional rounds proposed	INR 1,000 estimated
One additional round accepted	INR 500 client accepted
Invoice reference	Owner records an existing invoice after quoted requests are approved. No money is collected.

Workflow	Required action
1. Set policy	Invite a separate client; after access acceptance select their account, currency, allowance and price. Viewer access is sufficient for quote consent.
2. Propose extra work	Reviewer proposes a fixed quote covering exact request IDs, comparison wording and round scope.
3. Client response	The designated client accepts or declines from their signed-in account. Other accounts cannot consent on their behalf.
4. Scope approval	Reviewer approves separately. Current accepted quote supplies the separate agreement; a free-text note alone cannot bypass priced consent.
5. Changed scope	Request/proposal/group changes require a refreshed quote and renewed consent. Accepted prices stay fixed for that round.
6. Record invoice	Owner adds an invoice reference only after all quoted requests are approved. Invoiced rounds close to further expansion.

The meter separates unquoted estimates, current proposed quotes, client-accepted amounts and invoiced records. Currencies total separately. Accepted and invoiced totals overlap; do not add them. Pricing does not generate invoices, calculate tax, charge cards or send quote emails.

06 / Access, invitations and email

Role / capability	What the implementation allows
Owner	Controls project membership, deletion, revision pricing/policy, designated client and invoice references. Also reviews scope.
Reviewer	Approves baseline and scope decisions, edits proposals and proposes extra-round quotes.
Editor	Submits requests and manages tasks; cannot approve scope.
Viewer / client	Reads shared data and exports. A designated client with active access can accept/decline their own quote.
Invitation	Seven-day invite tied to the matching verified account. Pending invites grant no project access. Wrong-account acceptance is rejected; owner may revoke.
Unknown recipient	Owner gets an explicit confirmation option before inviting an account that has not signed up.
Address checks	Invalid syntax is rejected. A valid-looking address cannot be guaranteed to exist or belong to the intended person.
Welcome and access mail	Branded HTML with plain-text alternatives; welcome deduplication, invitations and delivery states are implemented. Mail is queued in persisted storage.

Current sender status: configured. This run tested mail/invitation logic, but did not send a fresh welcome or invitation email. A previously received styled preview proves only that earlier message's delivery.

The current mail worker uses a backend polling timer. A reliable hosted worker/scheduler is still needed for scale-to-zero deployment. SMTP acceptance does not guarantee inbox delivery.

Another person's localhost points to their computer. Remote collaboration requires the same reachable backend; a public deployment is still pending.

07 / Architecture and protections

Layer	Implementation and reason
Frontend	React, TypeScript and Vite provide the public site, workspace forms, comparisons, tour and task board.
Identity	Firebase Authentication supplies Google identity; backend token and verified-account checks enforce trust.
API / domain	Express + Zod validate inputs; backend membership/roles, evidence and decision rules enforce actions.
Persistence	Firebase Admin accesses scopesathi's default Firestore. Project aggregates include intake, brief, requirements, versions, requests, tasks, members, policy, quotes and history.
Analysis	Google Gen AI SDK supports Gemini Developer API in firebase mode and Vertex AI in google mode. Current installation is manual.
Mail	Nodemailer/Gmail SMTP and a persisted outbox support branded transactional messages.
Hosting	Local backend and Vite preview are running. Google public hosting configuration exists but is not deployed/verified.

Save path: browser ID token / API authentication / membership and workflow checks / Firestore transaction / confirmed response. A write failure is shown; there is no silent local fallback.

Revision checks prevent lost updates. Model calls occur outside transactions; their eventual save rechecks the original revision. Private Admin and mail/model credentials stay on the backend.

Rate limits, ingress budgets, request IDs, security headers, upload/JSON limits, HTTP timeouts and graceful shutdown are implemented. Rate counters are in-memory per instance; distributed limits, pagination and recovery operations remain scale work.

Projects are capped at 700 KB. Direct browser access is denied by repository Firestore rules; the deployed rule source was not inspected in this run. Backend outsider denial was tested.

08 / Validation results

Verification: 15 October 2026. Service checks used isolated or disposable fixtures; the live Firestore fixture was removed after verification.

Check	Result and boundary
Automated suite	46 of 46 tests passed. Covers baseline/quote evidence, multi-change review, ambiguity, roles, rejected access, concurrency, corrections, allowance, client quote consent, fixed rates, invoice prerequisites, invitation logic and API validation.
Build	TypeScript and Vite production build passed.
Real Firestore	Create project, baseline, request, quote, client acceptance, scope approval, invoice reference, task update, outsider denial and fresh-process reload passed.
Current runtime	Firebase mode; Firestore read connection verified; AI manual; mail sender configured. Both local web/API health checks passed.
Supplied PDF	Generated brief uploaded successfully: 1,381 text characters and 21 draft line entries. Seven reviewed requirements, three approved findings, baseline v2, linked task Needs Review and API reload passed in an isolated local fixture; no AI call occurred.
Fresh browser / mobile	Not verified in this run. Automatic browser security review blocked reloading the existing tab. Historical screenshots are labelled as such.
Real Google login / inbox	No fresh two-account Google login or new welcome/invitation delivery was executed. Unit/service evidence is not proof of these browser/inbox outcomes.
Live AI / public hosting	No live Gemini response or public deployment was verified.

Passing tests does not establish load capacity, model accuracy, legal agreement validity or production availability. The manual guide includes observations you should record rather than assume.

Evidence files: .data/submission-tests.log, .data/submission-build.log and .data/submission-firestore.log. These local logs are supporting records, not submission secrets.

09 / Pending work and submission readiness

Priority	What remains and how to verify
1. Live Gemini analysis	Configure private GEMINI_API_KEY and GEMINI_MODEL on the local backend, restart, then test genuinely unseen briefs and multi-change requests. Verify semantic matches and exact evidence; preserve human approval. No model accuracy results exist yet.
2. Public Google deployment	Resolve eligible hosting/billing/credits separately. Deploy the actual backend/frontend, authorize the hostname and verify sign-in, saves and reloads. GitHub source hosting alone does not run this API.
3. Two-account acceptance	Manually test Google sign-in, invitation, viewer restrictions, quote acceptance and reviewer approval using the guide.
4. Email delivery / worker	Check first-time welcome once, invite receipt and acceptance. Establish reliable hosted queue processing; verify failures/retry behavior and visible delivery states.
5. Browser regression	Complete desktop/mobile layout, loading, all sections, profile, tour, exports and unsaved-input failure checks.
6. Scale / operations	Add shared rate counters for replicas, pagination/subcollections, backup/restore, monitoring and retention policy before larger production use.

Not included: scanned-PDF OCR, automatic invoices/tax/payment collection, quote notification emails, third-party task integrations or general idempotency keys.

The core API is already implemented. The missing analysis dependency is the live model configuration and verification, not an absent request-management API.

Never paste a private key into chat or include it in frontend code. Use backend-only local environment configuration. Do not enable billing or migrate projects as part of these manual tests.

Submit this report with your source and evidence as appropriate. It does not replace a working deployed prototype if the event requires one. Documentation: README.md and docs/README.md link the architecture, data model, API, setup, security and revision-pricing guides.